

IOCP 네트워크 라이브러리 구축과 활용

조성찬

Phone: 010-5354-7426

Email: aa01053547426@gmail.com

Github: <https://github.com/ChokeOutCho>

목차

PART 1. IOCP 서버 네트워크 라이브러리 (Network Library Layer) 3

네트워크 어플리케이션의 신속한 개발에 도움을 주는 안전한 네트워크 라이브러리 설계

PART 2. IOCP 서버 네트워크 라이브러리 활용 (Application Layer) 14

네트워크 라이브러리의 생산성 증명

3종의 서버 어플리케이션 구축 (로그인 / 채팅 / 모니터링 서버)

PART 3. 기타 사항 24

3.1. 멀티스레드 환경에서 패킷 누수 원인 분석 및 해결

3.2. 락-프리 큐(Lock-Free Queue) 구현 타임라인

PART 1.

IOCP 서버 네트워크 라이브러리 (Network Library Layer)

목표

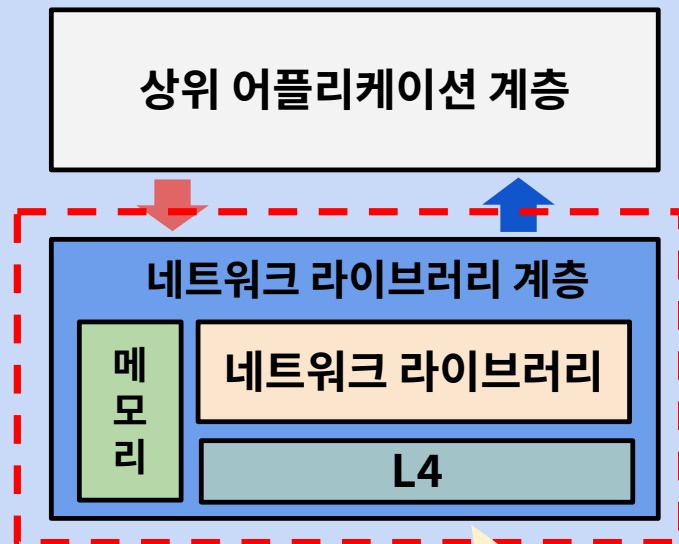
네트워크 어플리케이션의 신속한 개발에 도움을 주는
IOCP 기반의 범용 네트워크 라이브러리 개발

핵심 구성 요소

Accept 스레드 / 워커 스레드 / 세션 / 패킷 / TLS 오브젝트 풀

깃허브:

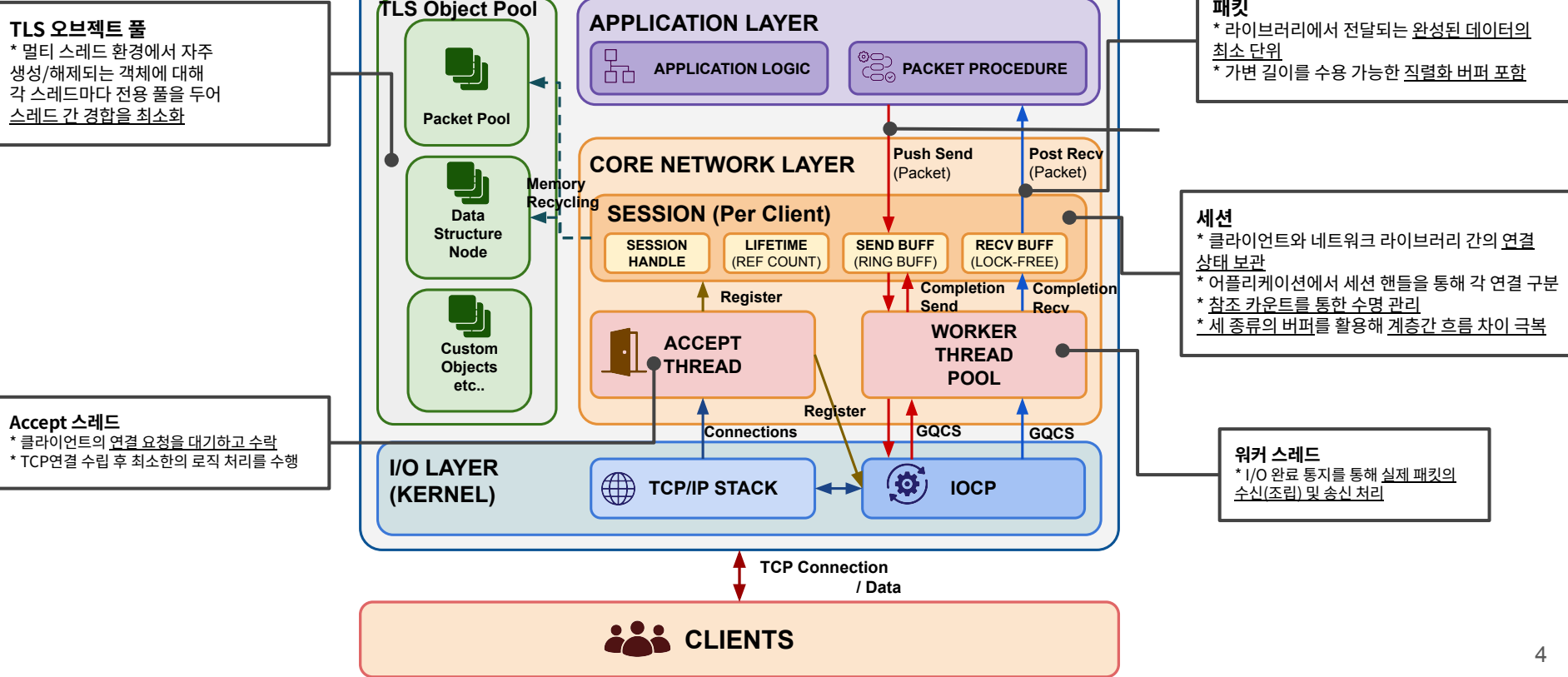
https://github.com/ChokeOutCho/IOCPNetModule_MonoRepo



네트워크 라이브러리에 필요한
핵심 요소들을 구축

PART1. 전체조망

IOCP 서버 네트워크 라이브러리



① 네트워크 라이브러리 클래스:

네트워크 통신과 세션 관리를 담당하는 상속용 기본 클래스.

사용자는 이 클래스를 상속받아 서버 애플리케이션(로그인, 게임, 채팅 등)을 구현할 수 있음.

네트워크 모듈 클래스 생성자

```
NetLib_Server(
    const WCHAR* openIP,
    unsigned short openPort,
    int opt_workerTH_pool_size,
    int opt_concurrentTH_size,
    int opt_maxOfSession,
    bool opt_zeroCopy,
    Opt_Encryption* opt_encryption,
    int opt_maxOfSendPackets);
```

매개변수	설명	비고
openIP	리슨 소켓에 바인드 할 IP 주소.	
openPort	리슨 소켓에 바인드 할 Port 번호.	
opt_workerTH_pool_size	IOCP에서 사용할 워커 스레드의 총 개수.	
opt_concurrentTH_size	IOCP에서 동시에 깨어날 수 있는 워커 스레드의 개수. (*※ opt_workerTH_pool_size 이하로 설정해야 함)	
opt_maxOfSession	네트워크 모듈에서 동시에 연결 가능한 세션의 최대 개수. (*※ 최대 65535까지 설정 가능)	
opt_zeroCopy	네트워크 모듈에서 송신 시 Zero-copy를 사용할지 여부를 결정하는 bool 값.	
opt_encryption	XOR 기반 난독화에 사용되는 구조체 변수. 클라이언트와 동일하게 설정해야 함. nullptr로 설정 시 패킷 난독화 옵션 비활성화.	<pre>struct Opt_Encryption { char Header_Code = 0x77; char Fixed_Key = 0x32; };</pre>
opt_maxOfSendPackets	송신 버퍼의 사이즈 제한(패킷 개수).	

② 네트워크 이벤트:

클라이언트의 접속 상태 변화나 데이터 수신 시 내부적으로 호출되는 콜백 메서드.
파생 클래스에서 목적에 맞게 오버라이딩하여 사용 가능.

메서드 명	파라미터	상세 설명
OnConnectionRequest	IP, port	클라이언트의 세션 연결 요청 시 호출 됨. - true 반환: 연결 수락 - false 반환: 연결 거부 (악성 IP 차단 및 L7 방어용)
OnClientJoin	sessionHandle, IP, Port	TCP 연결이 완료되고 IOCP에 세션이 바인딩된 직후 최초로 호출되는 입장 이벤트.
OnClientLeave	sessionHandle, code, IP, Port	세션 연결이 완전히 종료된 후 호출 됨. 모듈 계층에서 세션 연결을 끊어냈다면 code 값을 통해 사유를 파악할 수 있음.
OnRecv	sessionHandle, Packet*	클라이언트로부터 데이터 수신 완료되었을 때 호출되는 패킷 처리 창구.

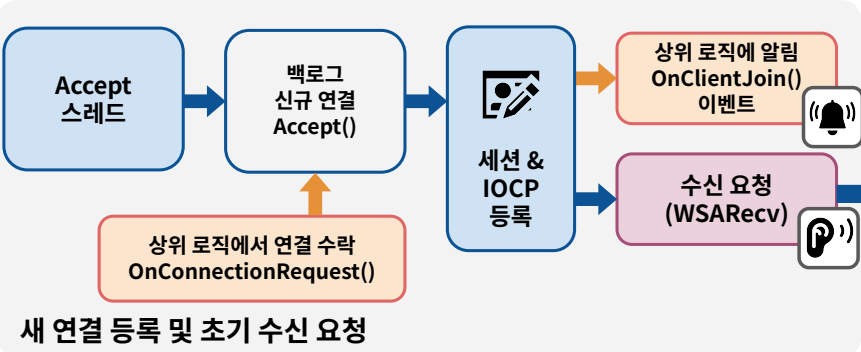
③ 요청 인터페이스:

연결된 세션을 제어하거나 데이터를 전송하기 위해 L7에서 직접 호출하는 메서드.

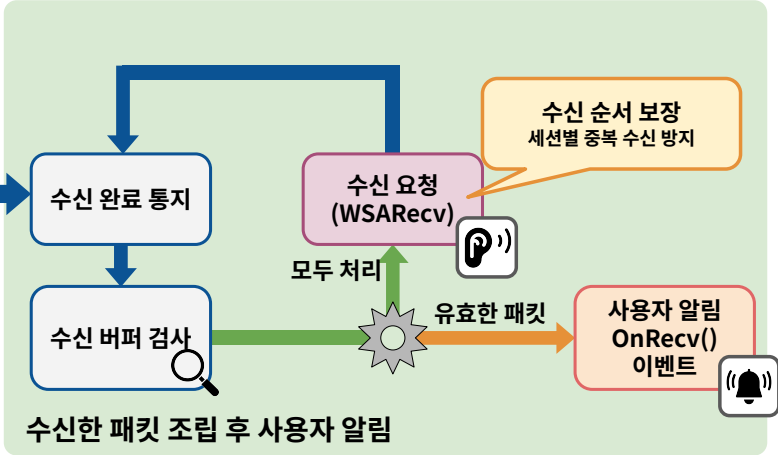
메서드 명	파라미터	상세 설명
SendPacket	sessionHandle, Packet*	특정 단일 세션에 직렬화된 패킷 객체를 비동기 송신 요청.
SendPacketMulticast	sessionsHandles[], Packet*	전달받은 다수의 세션 핸들 배열을 순회하며, 동일한 패킷을 여러 클라이언트에게 효율적으로 브로드캐스트/멀티캐스트.
Disconnect	sessionHandle	지정된 세션에 강제 연결 종료(Close)를 요청.

PART1. 전체 동작 방식

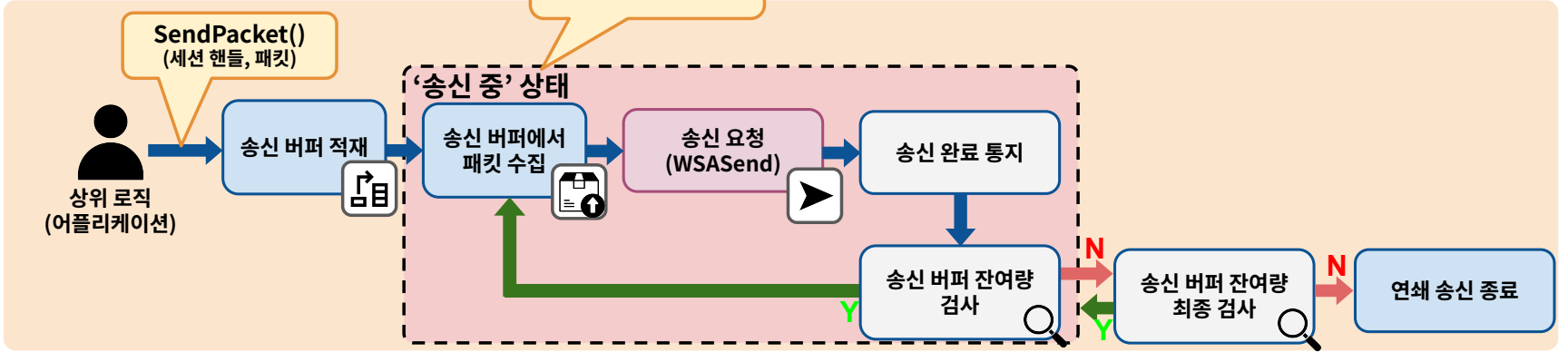
[동작 1. 최초 연결]

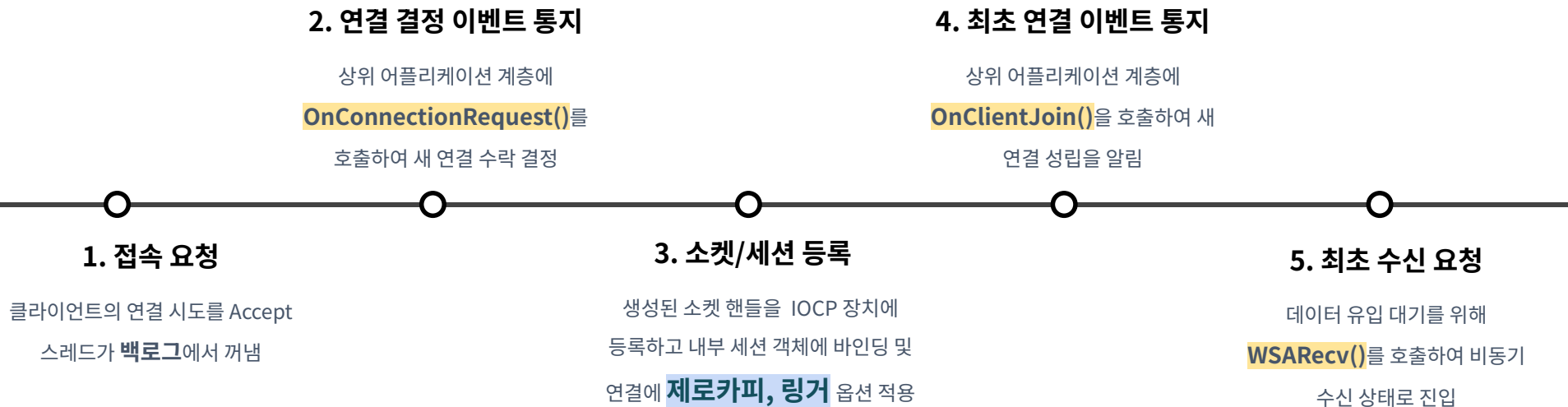


[동작 2. 수신]



[동작 3. 송신]





수신 완료 처리 (Recv)

수신한 데이터를 완성된 패킷 단위로 조립하여 상위 계층에 전달

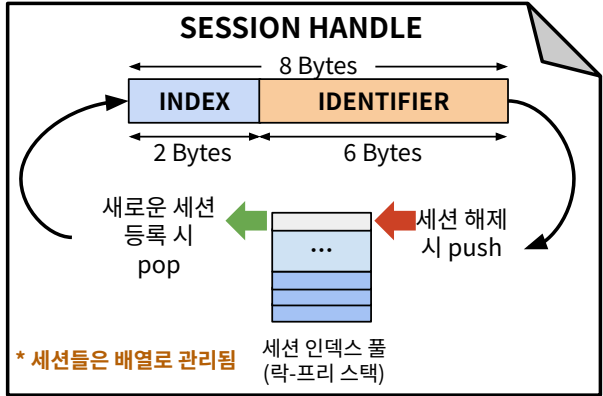
- (1) 완료 통지(**GQCS**) 확인 및 수신 링 버퍼 쓰기 포인터 갱신
- (2) 헤더 길이 검증을 통한 **유효 패킷** 조립 및 검사
- (3) 완성 시 **OnRecv()** 이벤트를 통한 상위 어플리케이션 전달
- (4) **WSARecv()** 재요청

송신 완료 처리 (Send)

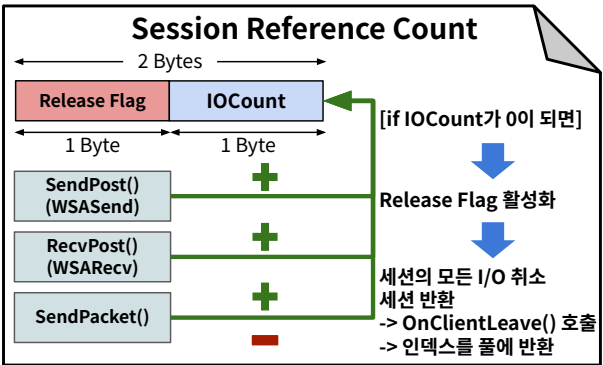
상위 어플리케이션의 큐잉 요청을 받아 비동기적으로 데이터를 전송하며, 연쇄적인 송신 흐름을 생성

- (1) 상위 어플리케이션에서 **SendPacket()** 호출 시 세션별 송신 버퍼에 **패킷 적재**
- (2) **진행 중인 송신 여부 플래그 확인** 후 **WSASend()** 호출
- (3) 송신 완료 시 **송신된 패킷 회수**
- (4) 송신 버퍼 잔여 패킷 확인 후 조건에 따른 **연쇄 재송신** 수행

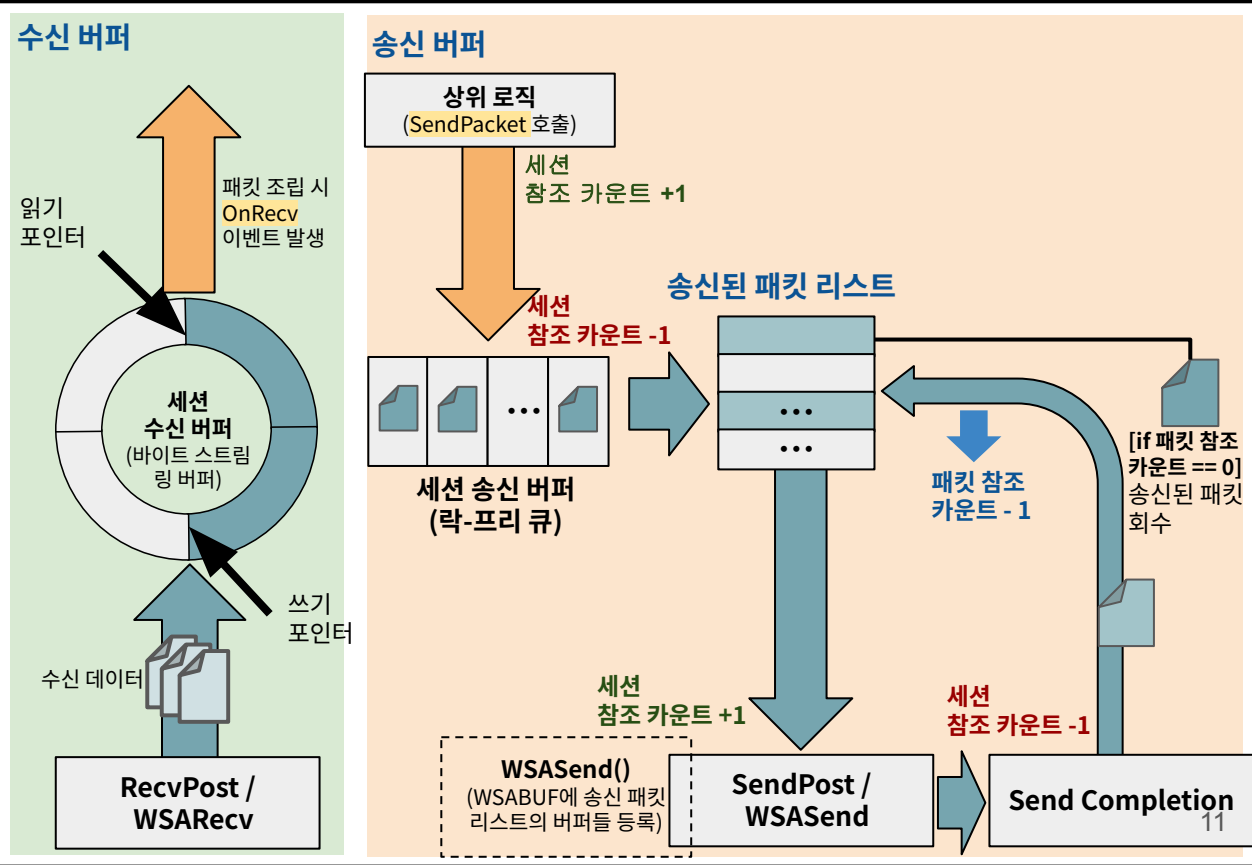
세션 핸들 & 인덱스 풀



세션 수명 관리



세션 버퍼



직렬화 버퍼

가변 길이 데이터를 바이트 배열 형태로 변환

(1) C++ iostream과 유사한

스트림 연산자 오버로딩 (<<, >>)을 통한
데이터의 직관적 Push/Pop 지원

(2) 가변 길이 구조체를 처리하기 위한 직접 데이터 메모리 복사 기능(PutData,
GetData) 제공

수명 관리 및 브로드캐스트 최적화

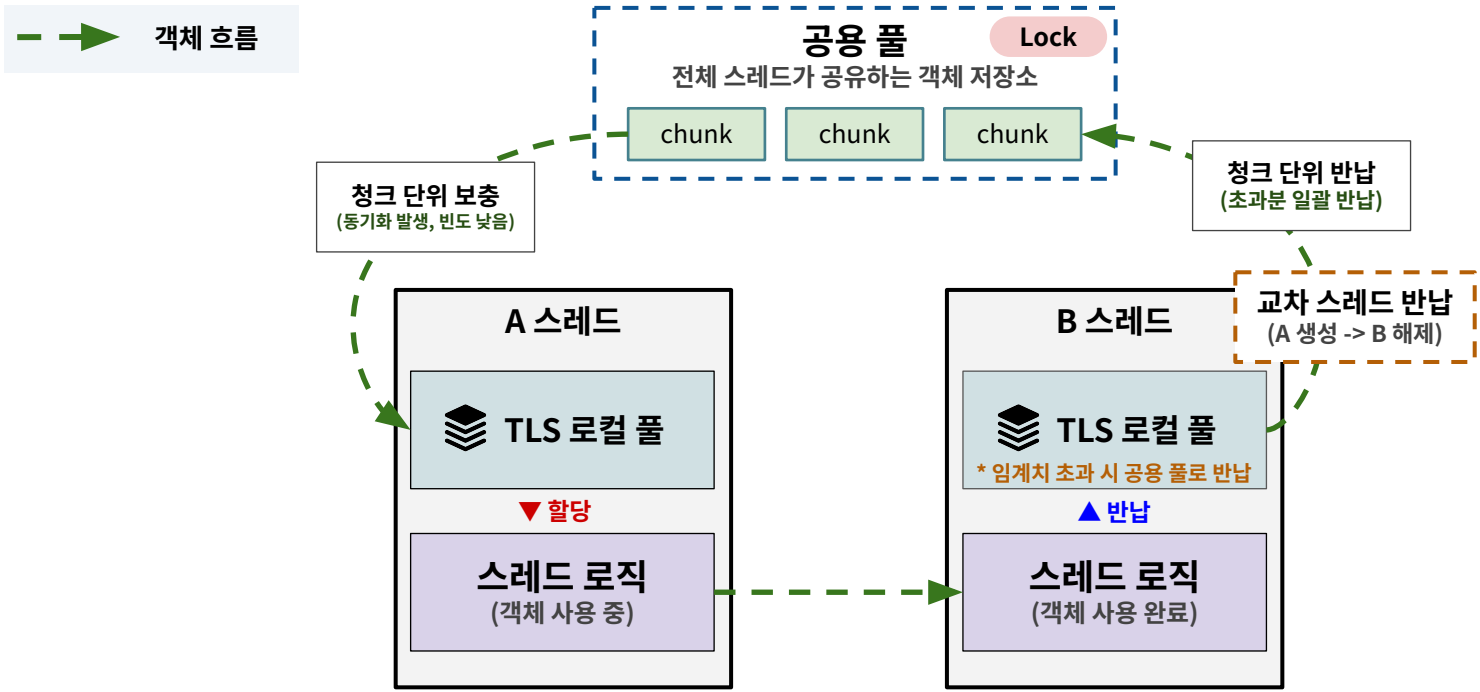
동일한 패킷을 여러 사용자에게 전달할 때 불필요한 복사 비용
절약을 위한 참조 카운트 기반 시스템

(1) 참조 카운트를 통해 생성(Alloc)부터 반환(Free)까지 패킷 객체 수명을
통제

(2) 동일 패킷 다수 세션 송신(브로드캐스트) 시 포인터와 참조 카운트만
공유

```
WORD messageLen;  
WCHAR message[MSG_LENGTH];  
unsigned long long accNo;  
  
// 패킷에서 데이터 Pop  
*packet >> accNo >> messageLen;  
packet->GetData((char*)message, sizeof(WCHAR)* messageLen);  
if (messageLen > MSG_LENGTH)  
{  
    server->Disconnect(sessionHandle);  
    LOG(L"DetectAttack_MSG", LEVEL_DEBUG, L"Detected attack - MSG_LENGTH: %d", messageLen);  
}  
  
// 풀에서 새로운 패킷 할당  
Packet* resPacket = Packet::Alloc();  
  
// 패킷에 데이터 Push  
*resPacket << (WORD)PACKET_CS_CHAT_RES_MESSAGE;  
*resPacket << player->AccNo;  
resPacket->PutData((char*)player->ID, sizeof(WCHAR) * ID_LENGTH);  
resPacket->PutData((char*)player->Nickname, sizeof(WCHAR) * NICKNAME_LENGTH);  
*resPacket << messageLen;  
resPacket->PutData((char*)message, sizeof(WCHAR) * messageLen);  
  
SendAroundSector(player, resPacket, true);  
  
// 풀에 패킷 반환  
Packet::Free(resPacket);  
player->lastRecv = currentTime;
```

▲ 상위 어플리케이션에서 패킷을 사용하는 예시



스레드 간 경합 제거
원자적 연산 비용을 소멸시켜 스레드 개수가 증가하더라도 성능 저하 곡선이 거의 발생하지 않음

자원 불균형 해결
교차 스레드 반납 시에도
체크 단위 배분배를 통해 균형 유지

주요 적용 대상
락-프리 자료구조 노드, 패킷 및
멀티 스레드에서 수명 주기가 짧은 기타 객체 전반

PART 2:

IOCP 서버 네트워크 라이브러리 활용 (Application Layer)

목표:

채팅 서버 시스템 아키텍처를 설계하고,
네트워크 라이브러리에서 제공하는 인터페이스를 활용해 각
구성요소를 신속하게 구축

핵심 구성 요소

로그인 서버 / 섹터 기반 채팅 서버 / 모니터링 서버 / 토큰 서버

깃허브:

https://github.com/ChokeOutCho/IOCPNetModule_MonoRepo

Part1에서 구축한
네트워크 라이브러리를 통해
서버 어플리케이션 개발





로그인 서버

MySQL 기반 계정 검증 및
인증 토큰 발급 프로세스
담당



채팅 서버

공간 상으로 나뉜
섹터내에만 채팅 메시지
브로드캐스트



토큰 서버

Redis를 활용한 서버 간
공통 인증 토큰 관리

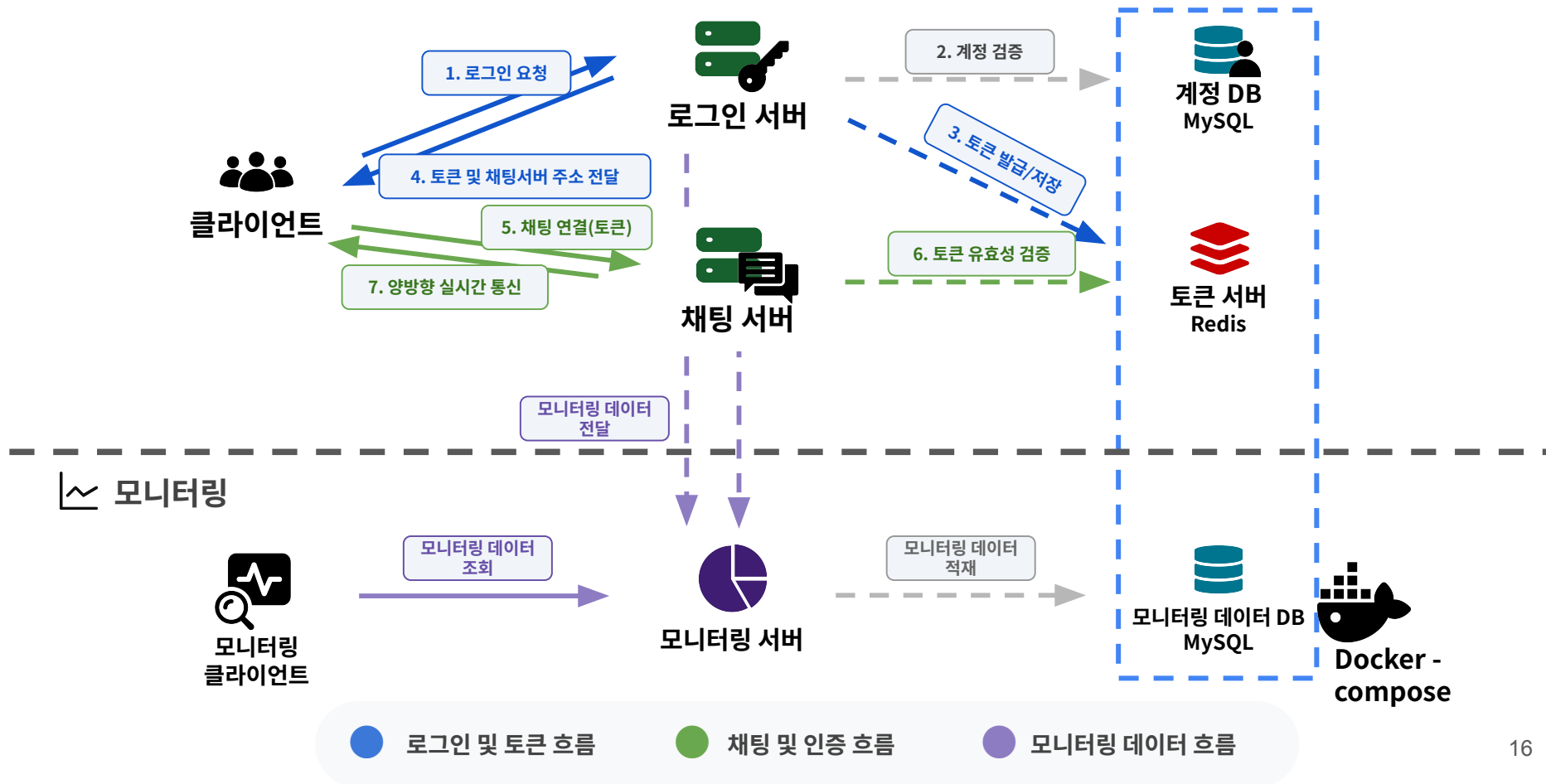


모니터링 서버

서버 상태 데이터 수집 및
모니터링 전용 MySQL DB
관리

PART2. 전체 조망

IOCP 서버 네트워크 라이브러리 활용

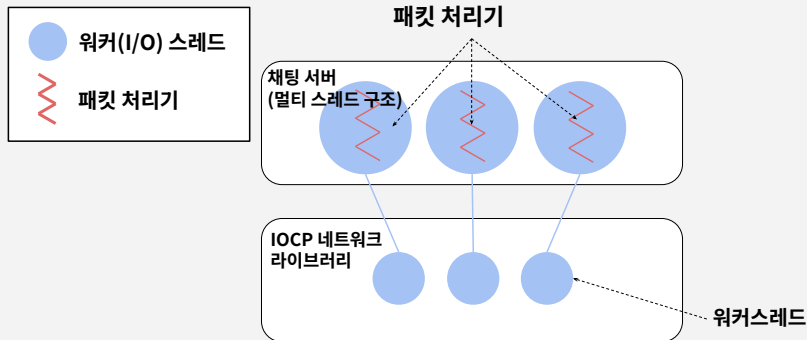


스레드 구조:

OnRecv 이벤트에서 즉시 처리

```
virtual void OnRecv(unsigned long long sessionHandle, Packet* packet)
{
    PROFILING("CONTENT");

    WORD contentType;
    *packet >> contentType;
    Content_Proc(sessionHandle, contentType, packet);
}
```



주요 기능1: 채팅 서버 로그인

정상 절차를 밟은 클라이언트 검증

- (1) 인증 전 세션을 별도 리스트로 고립 관리하고 인증 시간 내 인증 미완료 시 즉시 연결 해제
- (2) 토큰 서버에 접근해 클라이언트 토큰의 유효성 검증
- (3) 인증 완료 세션을 유저로 승격시키고 유저 자료구조에 등록

주요 기능2: 유저의 섹터 이동

단순 좌표 변경을 넘은, 자원 소유권의 이전을 야기하는 '쓰기'작업

- (1) 유저의 좌표가 변화할 때마다 현재 섹터ID를 계산
- (2) 유저가 섹터 경계를 넘으면 기존 섹터의 유저 리스트에서 제거하고, 새로운 섹터 리스트에 추가
- (3) 배타적(Exclusive) 락을 통해 섹터의 유저 리스트 무결성 확보

주요 기능3: 채팅 전파

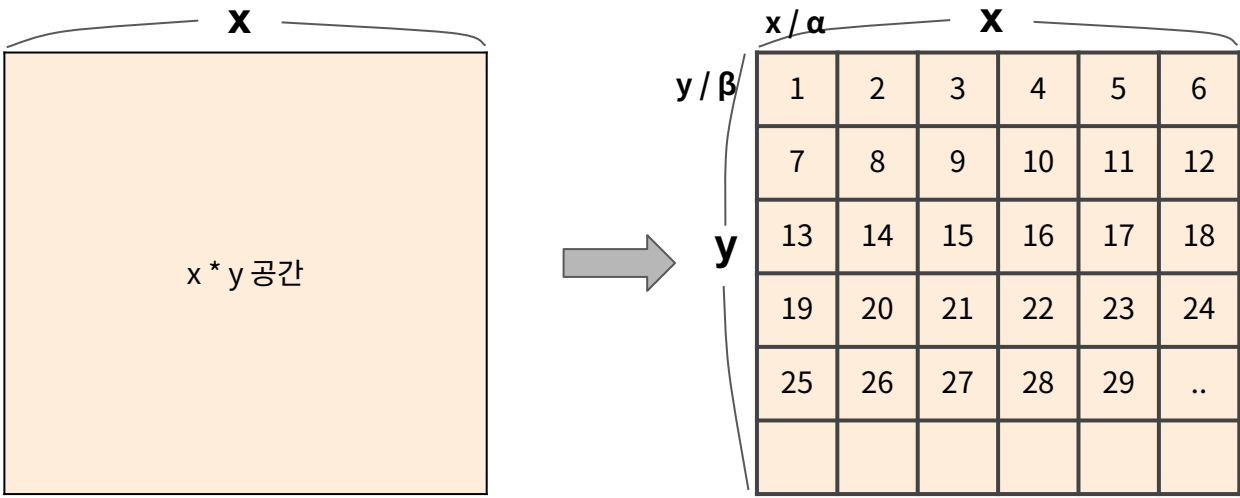
섹터 기반 시스템의 성능 이점을 극대화 하는 '읽기' 중심의 기능

- (1) 전체 접속자에게 메시지를 보내는 대신, 인접한 섹터의 유저 리스트만 순회
- (2) 섹터 리스트를 읽는 작업을 공유(Shared) 락을 통해 병렬 처리
- (3) 이동 작업(쓰기)이 일어나지 않는 한, 수많은 채팅 패킷이 동시에 브로드캐스트 될 수 있음

PART2. 섹터 확보 전략

- (1) OnRecv이벤트에서 패킷을 즉시 처리하는 스레드 구조를 택했기 때문에 공유 자원에 동기화 필요
-> 섹터는 주요 공유 자원임
- (2) 섹터 이동(주요기능2)과 채팅 전파(주요기능3)는 항상 둘 이상의 섹터를 한번에 취득해야 하기 때문에 별도의 섹터 확보 전략 필요

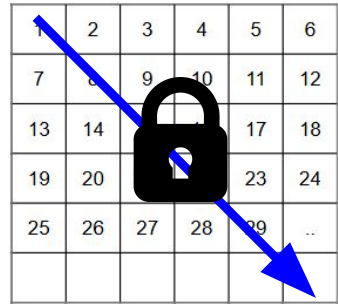
전체 공간을 섹터로 분할 후, 행 우선순으로 ID 부여



 유저는 좌표 기반으로 공간을 자유롭게 이동

 각 섹터에는 해당 섹터에 귀속된 유저 리스트가 존재

섹터 확보 순서



섹터 확보시에
섹터ID 오름차순으로 락을
취득하여 데드락 회피

ex) {20,3,7} 섹터 확보 필요 시
3 -> 7 -> 20 순으로 확보

이동 & 채팅 섹터 경합

→ 14번 섹터에 위치한 A유저의 이동(배타적 락)

16번 섹터에 위치한 B유저의 채팅(공유 락)

1	2	3	4	5	6
7	8	9	10	11	12
13		15		17	18
19	20	21	22	23	24
25	26	27	28	29	..

인접 섹터를
공유 락으로 취득

채팅 & 채팅 섹터 경합

8번 섹터에 위치한 A유저의 채팅(공유 락)

16번 섹터에 위치한 B유저의 채팅(공유 락)

1	2	3	4	5	6
7		9	10	11	12
13	14	15		17	18
19	20	21	22	23	24
25	26	27	28	29	..

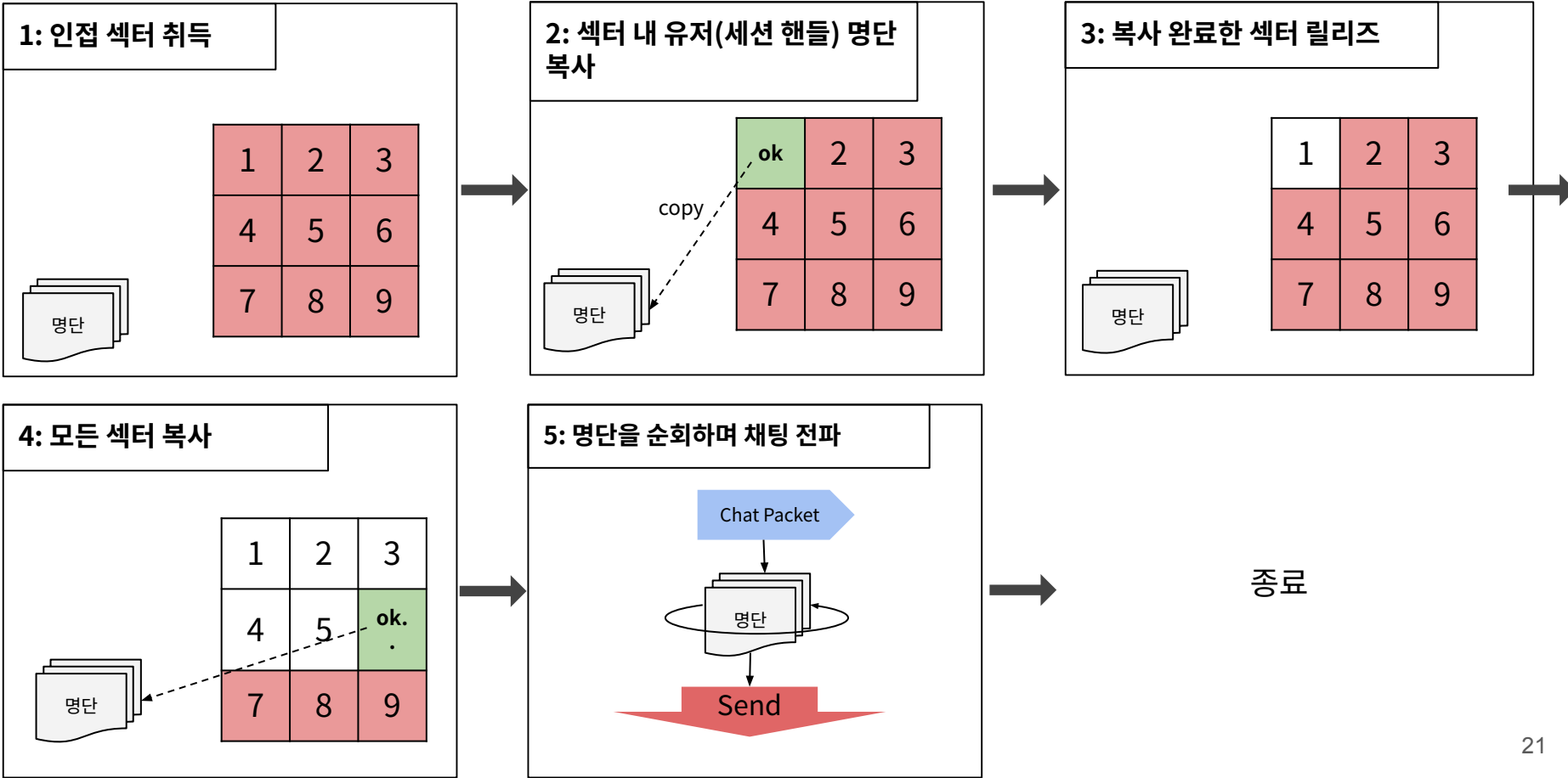
인접 섹터를
공유 락으로 취득

데드락 발생 X

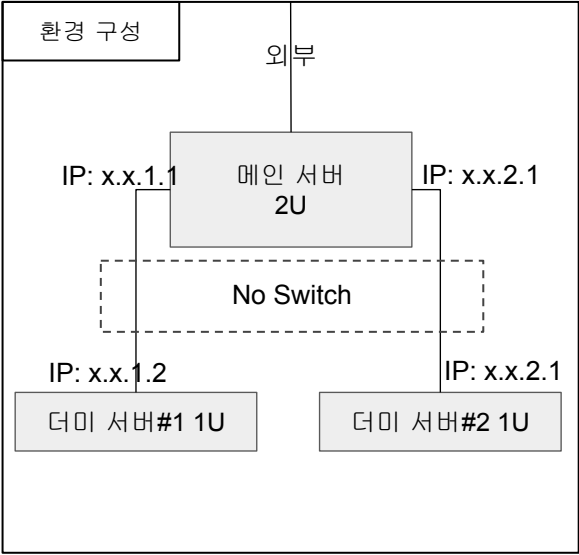
클라이언트가 채팅을 요청하는 경우

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	..

인접 섹터를
공유 락으로 취득



구분	항목	상세 사양	비고
메인 서버 (2U)	CPU	Intel(R) Xeon(R) CPU E5-2680 v4 @ 2.40GHz, 2401Mhz, 14 코어, 28 논리 프로세서	모든 서버 프로세스 가동
	RAM	16.0 GB	
	NIC #1	Realtek PCIe GBE Family Controller	
	NIC #2	Intel(R) I210 Gigabit Network Connection	더미 1 핫라인
	NIC #3	Intel(R) I210 Gigabit Network Connection #2	더미 2 핫라인
더미 서버 (1U * 2)	CPU	Intel(R) Core(TM) i5-4460 CPU @ 3.20GHz, 3201Mhz, 4 코어, 4 논리 프로세서	채팅 검증 더미 가동
	RAM	4.0 GB	
	NIC	Realtek PCIe GBE Family Controller (더미와의 이더넷 핫라인)	메인 서버와 1:1 직통
소프트웨어 환경	OS	Microsoft Windows Server 2019 (10.0.17763)	
	기타	C++17 (Visual Studio), 컴파일 최적화 미사용, __inline 확장 허용	



안정성 위주의 검증 조건 결정

항목	테스트 환경 및 조건	최종 결과	비고
장기 가동성	최대15,000개 더미 세션 동시 연결 및 초당 1200~1600개 세션 재접속 채팅 수신 초당 12000 ~ 16000 내외로 유지 168시간(7일) 가동	프로세스 다운 0건	
메시지 무결성	수십억 건의 섹터 브로드캐스트 패킷 송수신	데이터 변형 0건	송수신 데이터 1:1일치 확인
리소스 안정성	장기 가동 시 메모리 오염 및 포인터 오류 발생 여부, 메모리 누수	누수 및 오염 미발생	TLS 오브젝트 풀 시스템의 안정성
논리 무결성	빈번한 섹터 이동 및 섹터 브로드 캐스트	데드락 발생 0건	섹터 오름차순 확보 유효

PART 3. 기타 사항

PART 3.1. 패킷 누수 원인 분석 및 해결

PART 3.1. 패킷 누수 추적

[1. 이상 현상 감지]

서버 프로세스의 메모리 사용량이 계속해서 빠르게 증가함
패킷 전체 사용량이 함께 증가하는 것으로 패킷 누수를 인지

```
** CHAT SERVER **

Running Time:          47.02 sec

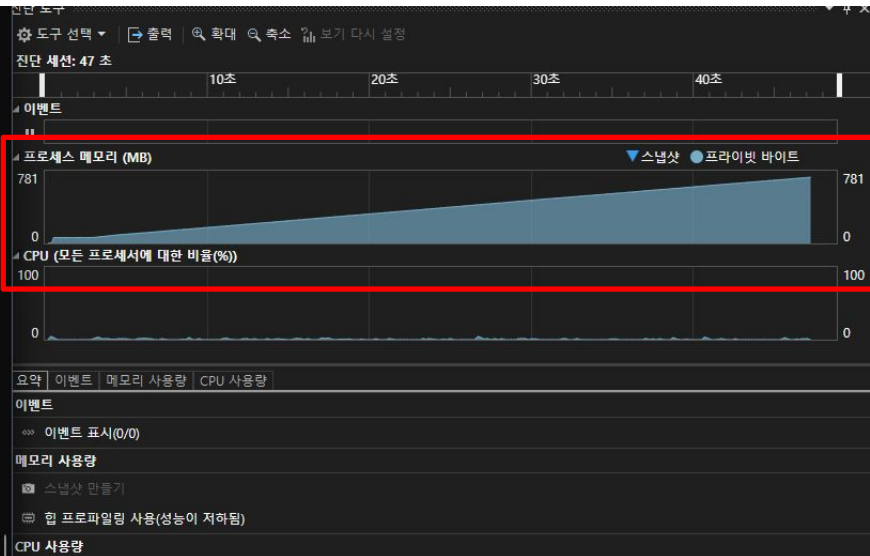
Monitor Server Connect: TRUE

Session Count:        400
Packet Pool Use:      437504
Total Accept:         5100
TPS_Accept:           59
TPS_Send:             4936
TPS_Recv:             3152
Waiting Queue:        63

Player Count Auth/Session: 337 / 337
Player Pool Use:        400

** Hardware **

ProcessorTotal:  5.11%   ProcessorUser:  3.13%   ProcessorKernel:  3.13%
ProcessTotal:   0.39%   ProcessUser:    3.13%   ProcessKernel:    0.31%
Available Mem:  15958 MB   Process Mem:    713 MB   NonPagedMem:      866 MB
Net Send:       2 KB/s    Net Recv:       0 KB/s
```



PART 3.1. 트러블슈팅: 패킷 누수 추적

[2. 추적]

- ① 패킷 객체 내부에 사용처 타입(Enum)을 삽입하여 생명주기를 기록
- ② 패킷 클래스에 디버그용 전역 리스트를 선언하고 패킷이 할당될 때 삽입, 해제될 때 제거

①

```
class Packet : public SerializeBuffer
{
public:
    enum class WhereUse
    {
        SENDPACKET,
        SENDPACK_MULTICAST,
        RECVPROC,
        LOGIN,
        SENDPOST,
        NETALLOC,
        ALLOC,
    };

    WhereUse type;
    inline static std::list<Packet*> debuglist;
    inline static SRWLOCK lock_debuglist;
```

②

```
__inline static Packet* Alloc()
{
    Packet* packet = pool.Alloc();
    packet->Clear();
    packet->refCount = 0;

    AcquireSRWLockExclusive(&lock_debuglist);
    debuglist.push_back(packet);
    ReleaseSRWLockExclusive(&lock_debuglist);
    return packet;
}

__inline static void Free(Packet* packet)
{
    AcquireSRWLockExclusive(&lock_debuglist);
    debuglist.remove(packet);
    ReleaseSRWLockExclusive(&lock_debuglist);
    pool.Free(packet);
}
```

PART 3.1. 트러블슈팅: 패킷 누수 추적

③ 서버 송수신을 일시 중지하고 잔류 객체를 분석한 결과 참조를 잃은 패킷 검출

- > 참조를 잃은 패킷들이 SENDPACKET 이후 실종됨.
- > 송신됐던 패킷이 완료통지에서 반환되지 못하고 사라짐.

debuglist	{ size=4694 }	③	std::list<Packet *,std::...
[allocator]	allocator		std::_Compressed_p...
[0]	0x000001d2732b7f10 {refCount=1 type=SENDPACKET (0) headerPtr=0x000001d27714b5c0 {Code=109 'm' Le...		Packet *
[1]	0x000001d2732b8320 {refCount=1 type=SENDPACKET (0) headerPtr=0x000001d2770fd080 {Code=119 'w' Le...		Packet *
[2]	0x000001d2732b7880 {refCount=1 type=SENDPACKET (0) headerPtr=0x000001d2771469e0 {Code=119 'w' Le...		Packet *
[3]	0x000001d277344760 {refCount=1 type=SENDPACKET (0) headerPtr=0x000001d277346070 {Code=119 'w' Le...		Packet *
[4]	0x000001d277344df0 {refCount=1 type=SENDPACKET (0) headerPtr=0x000001d2773455d0 {Code=109 'm' Le...		Packet *
[5]	0x000001d277344e90 {refCount=1 type=SENDPACKET (0) headerPtr=0x000001d2771d1510 {Code=109 'm' Le...		Packet *
[6]	0x000001d2773441c0 {refCount=1 type=SENDPACKET (0) headerPtr=0x000001d2771d1aa0 {Code=109 'm' Le...		Packet *

자동 조사식 1 로컬 스레드

④ 송신 버퍼에서 송신된 패킷 리스트로 패킷을 옮길 때, 아직 해제되지 않은 패킷들을 덮어쓰는 모습 발견 (m_sendPacketCount: 송신된 패킷리스트에 등록된 패킷 개수)

-> 완료통지가 완전히 처리되기 전에 중복으로 송신 요청이 진행됨

```
97 if (m_sendPacketCount != 0)
98 {
99     m_sendPacketCount 54
100     DebugBreak();
101 }
```

PART 3.1. 트러블슈팅: 패킷 누수 추적

[3. 원인 규명: 연쇄 송신 흐름의 중복 생성]

- ① 본래 송신 중 플래그(IsSending)로 한 세션에 대해서 유일한 송신 흐름을 보장하도록 설계
- ② 그러나 완료통지가 완전히 처리되기 전에 미세한 틈을 타 또 다른 송신 흐름이 생성되는 결함 확인
- ③ 이 과정에서 송신된 패킷 리스트에서 덮어쓰기 된 패킷들이 해제되지 못하고 유실됨

-> 연쇄 송신 흐름을 제어하는 메커니즘에 근본적인 재정립이 필요

GQCS 송신 완료 통지 처리

Race 처리 전 외부 송신 요청 침범

Result 이중 송신 흐름 생성

Sent(Pending) Packet List

OVERWRITE !!

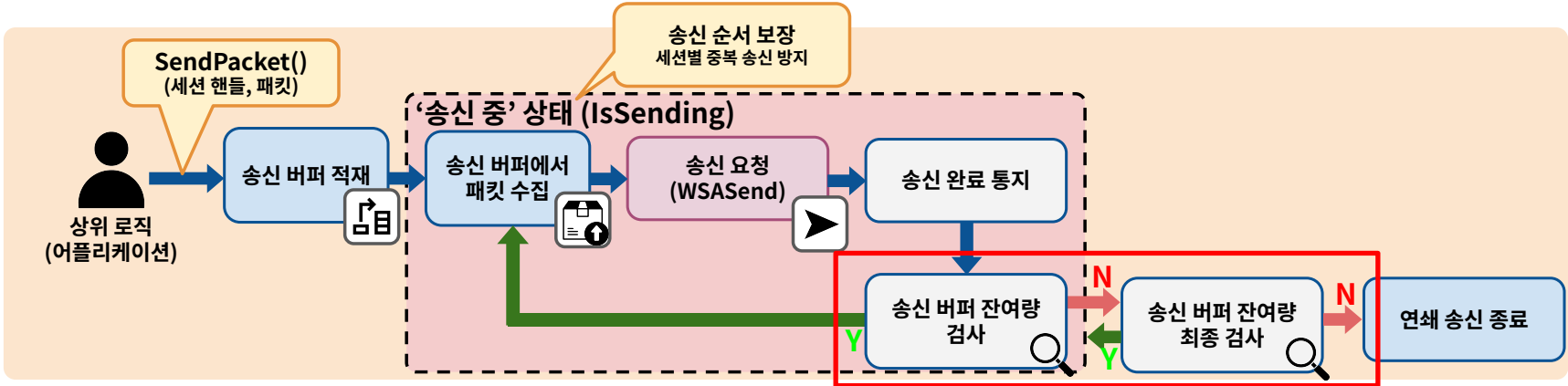
기존 패킷 포인터 덮어쓰기
-> 객체 해제 기회 상실

PART 3.1. 트러블슈팅: 패킷 누수 추적

[4. 해결: 연쇄 송신 흐름 재정립 (더블 체크링)]

- ① 권한 반납:
완료 통지에서 연쇄 송신 흐름이 종료되면 송신 중 플래그(IsSending)를 해제
- ② 연쇄 송신 흐름의 경쟁을 통한 안전한 재진입:
플래그 해제 직후, 다시 한번 송신 버퍼를 확인.
그 찰나에 외부 스레드가 데이터를 넣고 플래그를 획득하지 못했다면, 완료 통지 스레드가 다시 플래그를 획득하여 연쇄 송신 재개

기대 효과:
중복 송신을 차단하여 배열 덮어쓰기 현상을 원천적으로 제거
송신 완료와 동시에 데이터가 쌓이는 상황에서도 흐름이 끊기지 않고 즉시 연쇄 송신으로 이어짐



PART 3.1. 트러블슈팅: 패킷 누수 추적

[5. 최종 검증]

- ① 디버그용 리스트에 잔여 패킷 0건 확인
- ② 메모리 사용량 최대치 도달 후 24시간 동안 커밋 크기 변화 없음
- ③ PART 1&2의 테스트 결과에서 메모리 누수 미발생

1

	값	형식
debuglist	{ size=0 }	std::list<Packet *,std::allocator...
▶ [allocator]	allocator	std::_Compressed_pair<std::all...
▶ [Raw 뷰]	{ Mypair=allocator }	std::list<Packet *,std::allocator...

작업 관리자

파일(F) 옵션(O) 보기(V)

프로세스 성능 사용자 세부 정보 서비스

이름	PID	상태	사용자 이름	CPU	CPU 시간	메모리(활...	커밋 크기	NP 플	페이지 플트
fontdrvhost.exe	2016	실행 중	UMFD-2	00	0:00:00	4,176 K	5,000 K	8 K	3,924
LanServer.exe	2160	실행 중	행정실	03	15:44:53	90,052 K	154,736 K	49 K	23,434
LogonUI.exe	1316	실행 중	SYSTEM	00	0:01:00	9,416 K	12,776 K	26 K	4,171,857

검증에 사용된 머신의 프로세서는 4개,
프로세스 평균 CPU 점유율은 약 16%였음..

PART 3.2. 락-프리 큐 구현 타임라인

깃허브: <https://github.com/ChokeOutCho/LockFreeQueue> DevLog

PART 3.2. 락-프리 큐 구현 타임라인

[1 단계. ABA 방지와 접근 위반 해결]

문제:

락-프리 특성상 특정 스레드가 노드를 delete해도 다른 스레드가 해당 노드에 접근할 수 있음.
힙을 사용하면 페이지 회수 시점을 알 수 없음. 이로 인해 **페이지가 디커밋되면서 접근 위반 발생.**

해결:

노드 풀 도입하여 노드 생명 직접 관리

ABA 방지:

노드 할당 직후 상위 17비트에 할당 카운터 삽입

**풀에서 노드 할당 및
상위 17비트 카운터 삽입**

```
void Enqueue(T t)
{
    Node* newNode; // 새 노드 주소 + 상위 17비트 카운터
    Node* newNodeAdr; // newNode의 카운터를 마스크한 주소
    Node* nextCap; // next 노드 주소 캡처 + 카운터
    Node* nextCapAdr; // nextCap의 카운터를 마스크한 주소
    Node* tailCap; // tail 주소 + 상위 17비트 카운터
    Node* tailCapAdr; // tail의 카운터를 마스크한 주소
    Node* oldTail;

    m_sNodePoolLock.lock();
    newNode = m_sNodePool.Alloc();
    m_sNodePoolLock.unlock();

    unsigned long tailCnt = InterlockedIncrement(&m_allocCnt);

    newNode->data = t;
    newNode->next = nullptr;
    newNode = (Node*)SetCount(newNode, tailCnt);
}
```

카운터 삽입 함수

```
__inline unsigned long long SetCount(Node* target, unsigned long long count)
{
    unsigned long long ret = (unsigned long long)target & ADR_MASK;
    count = count << 47;
    ret = ret | count;
    return ret;
}
```

풀로 노드 반환

```
t = nextCapAdr->data; // 데이터를 미리 카피되어야됨 CAS성공하고 뜨면 변할 수 있음
if (InterlockedCompareExchangePointer((PVOID*)&m_head, nextCap, headCap) == headCap)
{
    m_sNodePoolLock.lock();
    m_sNodePool.Free(headCapAdr);
    m_sNodePoolLock.unlock();
    break;
}
```

PART 3.2. 락-프리 큐 구현 타임라인

[2 단계. 메모리 로깅을 통한 문제 추적 1]

문제:

인큐 도중 무한루프에 갇히는 현상 발생.

추적 도구:

파일 I/O를 사용한 로깅은 미세한 경합을 재현하기에 적합하지 않음

큐의 동작을 분석하기 위해 메모리 로깅 클래스 설계

문제: 무한루프 발생

```

79 while (true)
80 {
81     tailCap = m_tail;
82     tailCapAdr = (Node*)(unsigned long long)tailCap & ADR_MASK;
83     nextCap = tailCapAdr->next;
84     nextCapAdr = (Node*)(unsigned long long)nextCap & ADR_MASK;
85     if (nextCapAdr == nullptr) 경과 시간 1ms 이하
86     {
87         nextCapAdr = 0x0000027053172af0 (data=0x00000000 next=0);
88         if (InterlockedCompareExchangePointer((PVOID*)&tailCapAdr->next, nextCapAdr, nextCapAdr) != nextCapAdr)
89         {
90             break;
91         }
92     }
93     oldTail = (Node*)InterlockedCompareExchangePointer((PVOID*)&m_tail, newNode, oldTail);
94     InterlockedIncrement(&m_size); // 인구가 완전히 정리되면 사이즈 올림
95 }
96

```

추적 도구: 메모리 로깅 클래스

```

class LockFreeQueue_MemLog
{
public:
    const unsigned long long ADX_MASK = 0b0000000000000000011111111111111111111111111111111111111111111111;

    LockFreeQueue_MemLog()
    {
        indexPool = 0;
        lastUpdateIndex = 0;
        memset(samples, 0, sizeof(LockFreeQueue_MemLog_Sample) * LOCKFREEQUEUE_MEMLOG_SAMPLES_SIZE);
    }

    enum LockFreeQueue_DebugEvent
    {
        ENQ_CAS_1_SUCC,
        ENQ_CAS_1_FAIL,
        ENQ_CAS_2_SUCC,
        ENQ_CAS_2_FAIL,
        DEQ_CAS,
        DEQ_FREE,
    };

    struct LockFreeQueue_MemLog_Sample
    {
        LockFreeQueue_DebugEvent Event;
        unsigned long ThreadID;
        long Cnt;
        LockFreeQueue_Node<T>* TailCap; // ENQ에서 tail값을 캡처해둔 지역변수
        LockFreeQueue_Node<T>* TailCapAdr; // 캡처한 tail값의 진짜 주소
        LockFreeQueue_Node<T>* OldTail; // ENQ에서 CAS_2가 반환한 tail의 값
        LockFreeQueue_Node<T>* NextCap; // 이게 0이면 CAS_1이 성공함
        LockFreeQueue_Node<T>* NewTail; // ENQ로 인해 tail이 될 주소

        LockFreeQueue_Node<T>* HeadCap; // DEQ에서 head값을 캡처해둔 지역변수 (이번 DEQ에서 플로 반환됨)
        LockFreeQueue_Node<T>* HeadCapAdr; // DEQ에서 반환하는 더미노드의 진짜 주소
        long Size;
    };

    LockFreeQueue_MemLog_Sample samples[LOCKFREEQUEUE_MEMLOG_SAMPLES_SIZE]; // 샘플 배열. 인터락으로 인덱스를 부여받아서 쓰기
    long indexPool; // 인덱스를 부여할 때 인터락 대상
    long lastUpdateIndex; // 가장 마지막으로 갱신된 인덱스

    void Write_ENQ_CAS_1(LockFreeQueue_Node<T>* tailCap, LockFreeQueue_Node<T>* newTail, bool success){...,}
    void Write_ENQ_CAS_2(LockFreeQueue_Node<T>* tailCap, LockFreeQueue_Node<T>* oldTail, LockFreeQueue_Node<T>* newTail, bool success){...,}
    void Write_DEQ_NO_FREE(LockFreeQueue_Node<T>* headCap){...,}
};

```

PART 3.2. 락-프리 큐 구현 타임라인

[2 단계. 메모리 로깅을 통한 문제 추적 1]

추적 1: 무한 루프로 빠진 원인 규명

이 락-프리큐는 다음과 같은 연산을 기반으로 함.

- 1) tail의 next에 새로운 노드 연결. 이하 **CAS_1**.
- 2) tail을 새로운 삽입된 노드로 갱신. 이하 **CAS_2**.

마지막 로그에서 CAS_2 실패 확인

CAS_2가 실패해 tail이 갱신되지 않으면서 무한루프로 진입함

CAS_2가 실패한 원인 추적 시작

0x000016a4	{Event=ENO_CAS_2_FAIL (0x00000003) ThreadID=0x000060f4 Cnt=0x001f1054 ...}	LockFreeQueue_MemLog<int...
Event	ENO_CAS_2_FAIL (0x00000003)	LockFreeQueue_MemLog<int...
ThreadID	0x000060f4	unsigned long
Cnt	0x001f1054	long
TailCap	0x2aa601f9e27148b0 {data=??? next=??? }	LockFreeQueue_Node<int> *
TailCapAdr	0x000001f9e27148b0 {data=0x00000003 next=0x2aa681f9e27147f0 {data=??? next=??? } }	LockFreeQueue_Node<int> *
data	0x00000003	int
next	0x2aa681f9e27147f0 {data=??? next=??? }	LockFreeQueue_Node<int> *
OldTail	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
NextCap	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
NewTail	0x2aa681f9e27147f0 {data=??? next=??? }	LockFreeQueue_Node<int> *
HeadCap	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
HeadCapAdr	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
Size	0x00000000	long

PART 3.2. 락-프리 큐 구현 타임라인

[2 단계. 메모리 로깅을 통한 문제 추적 1]

추적 2: 메모리 로그 분석 - 재사용 문제 확인

- ① 스레드 A(0x000060f4)에서 인큐하기 위해 현재 tail인 노드 α (0x000001f9e27148b0)를 지역에 캡처
- ② 스레드 B(0x00007558)에서 노드 α 를 디큐

0x00001697	{Event=DEQ_FREE (0x00000005) ThreadID=0x00007558 Cnt=0x001f1047 ...}	LockFreeQueue_MemLog<int...
Event	DEQ_FREE (0x00000005)	LockFreeQueue_MemLog<int...
ThreadID	0x00007558	unsigned long
Cnt	0x001f1047	long
TailCap	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
TailCapAdr	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
OldTail	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
NextCap	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
NewTail	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
HeadCap	0x2ace81f9e27148b0 {data=??? next=???}	LockFreeQueue_Node<int> *
HeadCapAdr	0x000001f9e27148b0 {data=0x00000003 next=0x2aa681f9e27147f0 {data=??? next=???}}	LockFreeQueue_Node<int> *
Size	0x00000000	long

노드 α 디큐

- ③ 이후 스레드 B가 노드 α 를 재사용

0x000016a1	{Event=ENQ_CAS_1_SUCC (0x00000000) ThreadID=0x00007558 Cnt=0x001f1051 ...}	LockFreeQueue_MemLog<int...
Event	ENQ_CAS_1_SUCC (0x00000000)	LockFreeQueue_MemLog<int...
ThreadID	0x00007558	unsigned long
Cnt	0x001f1051	long
TailCap	0x2ad501f9e27145e0 {data=??? next=???}	LockFreeQueue_Node<int> *
TailCapAdr	0x000001f9e27145e0 {data=0x00000002 next=0x2ad581f9e27148b0 {data=??? next=???}}	LockFreeQueue_Node<int> *
OldTail	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
NextCap	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
NewTail	0x2ad581f9e27148b0 {data=??? next=???}	LockFreeQueue_Node<int> *
HeadCap	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
HeadCapAdr	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
Size	0x00000000	long

노드 α 재사용되어 인큐됨

PART 3.2. 락-프리 큐 구현 타임라인

[2 단계. 메모리 로깅을 통한 문제 추적 1]

- ④ 스레드 A는 노드 α 의 재사용을 인지하지 못하고 인큐를 진행했음.
(캡처했던 α 의 next는 여전히 nullptr이기 때문에 CAS_1은 성공함) 이후 상위17비트가 달라 CAS_2 실패

0x00000000	{Event=ENO_CAS_1_SUCC (0x00000000) ThreadID=0x000060f4 Cnt=0x001f1052 ...}	long
[0x000016a2]	ENQ_CAS_1_SUCC (0x00000000)	LockFreeQueue_MemLog<int...
Event		LockFreeQueue_MemLog<int...
ThreadID	0x000060f4	unsigned long
Cnt	0x001f1052	long
TailCap	0x2aa601f9e27148b0 {data=??? next=??? }	LockFreeQueue_Node<int> *
TailCapAdr	0x000001f9e27148b0 {data=0x00000003 next=0x2aa681f9e27147f0 {data=??? next=??? } }	LockFreeQueue_Node<int> *
OldTail	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
NextCap	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
NewTail	0x2aa681f9e27147f0 {data=??? next=??? }	LockFreeQueue_Node<int> *
HeadCap	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
HeadCapAdr	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
Size	0x00000000	long
[0x000016a2]	{Event=ENO_CAS_2_SUCC (0x00000000) ThreadID=0x00007558 Cnt=0x001f1052 ...}	LockFreeQueue_MemLog<int...

노드 α 를 여전히 tail로 보고 인큐 진행

해결: CAS_2의 실패를 인지하고 인큐 시 tail을 갱신할 수 있도록 변경

```
// tail을 밀어낼 수 있으면 밀기
if (nextCapAdr != nullptr)
{
    InterlockedCompareExchangePointer((PVOID*)&m_tail, nextCap, tailCap);
    continue;
}
```


PART 3.2. 락-프리 큐 구현 타임라인

[3 단계. 메모리 로깅을 통한 문제 추적 2]

추적 1:

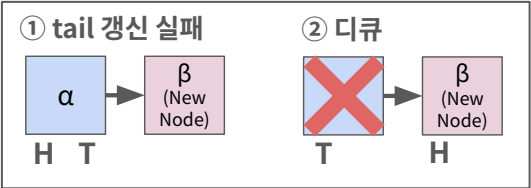
- ① 새로운 노드 β를 삽입한 후 tail 갱신(CAS_2)이 실패했지만 사이즈는 증가
이때 노드α(0x0000020eb21b4ac0)는 큐의 Head이자 Tail
- ② 사이즈가 존재하기 때문에 디큐 가능한 것으로 판단하고 디큐

[0x000024e]	{Event=ENQ_CAS_2_FAIL (0x00000003) ThreadID=0x00008c84 Cnt=0x163e4436 ...}	LockFreeQueue_MemLog<int...
Event	ENQ_CAS_2_FAIL (0x00000003)	LockFreeQueue_MemLog<int...
ThreadID	0x00008c84	unsigned long
Cnt	0x163e4436	long
TailCap	0xa2ed820eb21b4ac0 {data=??? next=???}	LockFreeQueue_Node<int> *
TailCapAdr	0x0000020eb21b4ac0 {data=0x00000000 next=0xa306820eb21b4ac0 {data=??? next=???}}	LockFreeQueue_Node<int> *
OldTail	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
NextCap	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
NewTail	0xa2ee020eb21b45b0 {data=??? next=???}	LockFreeQueue_Node<int> *
HeadCap	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
HeadCapAdr	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
Size	0x00000000	long

노드β tail 갱신 실패

[0x000024d]	{Event=DEQ_FREE (0x00000005) ThreadID=0x00008c84 Cnt=0x163e443d ...}	LockFreeQueue_MemLog<int...
Event	DEQ_FREE (0x00000005)	LockFreeQueue_MemLog<int...
ThreadID	0x00008c84	unsigned long
Cnt	0x163e443d	long
TailCap	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
TailCapAdr	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
OldTail	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
NextCap	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
NewTail	0x0000000000000000 <NULL>	LockFreeQueue_Node<int> *
HeadCap	0xa306020eb21b4ac0 {data=??? next=???}	LockFreeQueue_Node<int> *
HeadCapAdr	0x0000020eb21b4ac0 {data=0x00000000 next=0xa306820eb21b4ac0 {data=??? next=???}}	LockFreeQueue_Node<int> *
Size	0x00000000	long

노드α가 Tail이자 Head이지만 디큐 성공



PART 3.2. 락-프리 큐 구현 타임라인

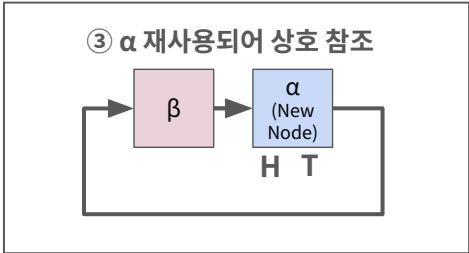
[3 단계. 메모리 로깅을 통한 문제 추적 2]

추적 2:

③ 이후 디큐했던 노드 α의 재사용-인큐로 인해 큐 붕괴(노드 α와 노드 β가 서로를 가리킴)

0x000024ee	{Event=ENQ_CAS_1_SUCC (0x00000000) ThreadID=0x00008c84 Cnt=0x163e443e ...}
Event	ENQ_CAS_1_SUCC (0x00000000)
ThreadId	0x00008c84
Cnt	0x163e443e
TailCap	0xa306020eb21b4ac0 {data=??? next=??? }
TailCapAdr	0x0000020eb21b4ac0 {data=0x000000000 next=0xa306820eb21b4ac0 {data=??? next=??? } }
OldTail	0x0000000000000000 <NULL>
NextCap	0x0000000000000000 <NULL>
NewTail	0xa306820eb21b4ac0 {data=??? next=??? }
HeadCap	0x0000000000000000 <NULL>
HeadCapAdr	0x0000000000000000 <NULL>
Size	0x00000000
0x000024ef	{Event=ENQ_CAS_2_SUCC (0x00000002) ThreadID=0x00008c84 Cnt=0x163e443f ...}
Event	ENQ_CAS_2_SUCC (0x00000002)
ThreadId	0x00008c84
Cnt	0x163e443f
TailCap	0xa306020eb21b4ac0 {data=??? next=??? }
TailCapAdr	0x0000020eb21b4ac0 {data=0x000000000 next=0xa306820eb21b4ac0 {data=??? next=??? } }
OldTail	0x0000000000000000 <NULL>
NextCap	0x0000000000000000 <NULL>
NewTail	0xa306820eb21b4ac0 {data=??? next=??? }
HeadCap	0x0000000000000000 <NULL>
HeadCapAdr	0x0000000000000000 <NULL>
Size	0x00000000

노드 α 다시 인큐됨



해결: CAS_2의 실패를 인지하고 디큐에서 head==tail일 때 tail을 갱신하도록 변경

```
// CAS_2 실패하고 head==tail일때 디큐하면 큐 붕괴함
// tail 밀어주기
if (headCap == tailCap)
{
    InterlockedCompareExchangePointer(&PVOID*, &m_tail, nextCap, tailCap);
    continue;
}
```

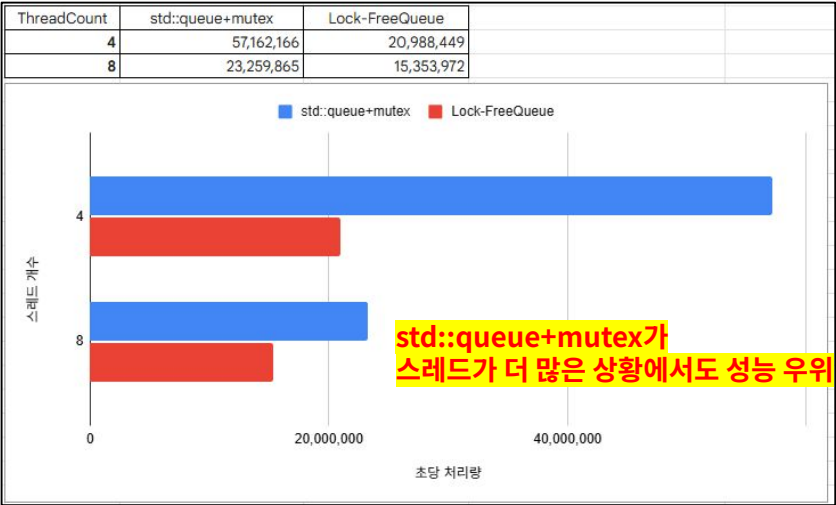
PART 3.2. 락-프리 큐 구현 타임라인

[4 단계. std::queue와의 성능 비교 (TLS 노드 풀을 통한 경합 제거)]

비교 환경: std::queue+mutex와 MPMC환경에서 1024바이트 데이터를 총 1억 회 인큐- 1억 회 디큐. (평균 TPS 비교)
내용:

예상과는 달리 현재의 락-프리 큐의 초당 처리량이 std::queue + mutex보다 현저히 떨어짐
프로파일링 결과 **노드 풀에 접근하는 비용**이 락-프리 큐 동작 시간의 **약 60%차지**

-> 노드 풀에 대한 경합을 병목 지점으로 판단하고 TLS를 사용해 경합 제거 시도



프로파일링 결과
(ENQUEUE < ALLOC
DEQUEUE < FREE)

ThreadID	Name	Average	Min	Max	Call
16328	ALLOC	0.4079µs	0.0000µs	3832.5000µs	125000000
16328	ENQUEUE	0.6281µs	0.0000µs	3832.9000µs	125000000
16328	FREE	0.4503µs	0.0000µs	3838.1000µs	125000000
16328	DEQUEUE	0.6745µs	0.0000µs	3838.5000µs	125000000

노드 할당과 반환이
전체 동작 시간의 약 60%차지

PART 3.2. 락-프리 큐 구현 타임라인

[4 단계. std::queue와의 성능 비교 (TLS 노드 풀을 통한 경합 제거)]

적용 결과:

기존의 락-프리 큐에서 **약 54% 성능 개선**됨.
이제 스레드가 많아질 수록 std::queue+mutex보다 적은 성능 하락을 보이며,
스레드 8개에서 초당 처리량이 역전됨

스레드가 많아질 때
mutex보다 적은 성능 하락 보임

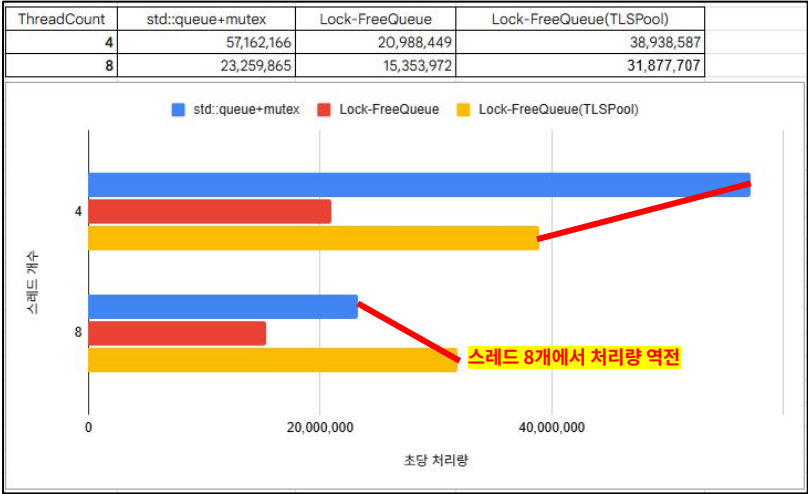
기존 락-프리 큐

ThreadID	Name	Average	Min	Max	Call
16328	ALLOC	0.4079µs	0.0000µs	3832.5000µs	125000000
16328	ENQUEUE	0.6281µs	0.0000µs	3832.9000µs	125000000
16328	FREE	0.4503µs	0.0000µs	3838.1000µs	125000000
16328	DEQUEUE	0.6745µs	0.0000µs	3838.5000µs	125000000

TLS 노드 풀 적용된 락-프리 큐

ThreadID	Name	Average	Min	Max	Call
23940	ALLOC	0.0376µs	0.0000µs	980.3000µs	125000000
23940	ENQUEUE	0.3242µs	0.0000µs	1571.9000µs	125000000
23940	FREE	0.0197µs	0.0000µs	697.4000µs	125000000
23940	DEQUEUE	0.2618µs	0.0000µs	3985.4000µs	125000000

전체 동작 시간 기존보다 약 54% 개선



회고:

하나의 큐에 대해 많은 경합이 발생하면 mutex의 문맥교환 비용보다 락-프리 큐의 스핀락과 유사한 CAS가 더 저렴한 경향을 보임.
그러나 실제 개발에서 동기화 수단으로 큐를 적용한 영역은 경합이 거의 발생하지 않았음.
개발 난이도와 리스크를 고려하면 굳이 락-프리 큐를 적용할 필요를 느끼지 못할 뿐더러,
대부분 작업의 직렬화를 기대하고 큐를 적용하기 때문에 이러한 락-프리 큐 대신 MPSC에 강력한 큐를 따로 고안하는 편이 좋아보임.

감사합니다

조성찬

Phone: 010-5354-7426

Email: aa01053547426@gmail.com

Github: <https://github.com/ChokeOutCho>